Name: Vivaan Balchandani

Roll No: 2301007

Batch: T11

Experiment 5

Aim: To build the pipeline of jobs using Maven / Gradle / Ant in Jenkins, create a pipeline script to test and deploy an application over the tomcat server.

Theory:

1. Introduction to CI/CD Orchestration

The modern software development lifecycle (SDLC) emphasizes Continuous Integration (CI) and Continuous Deployment (CD) to reduce the time between writing code and delivering it to production. In this laboratory experiment, Jenkins serves as the central orchestration engine.

Continuous Integration involves the automated merging of code changes into a central repository, followed by automated builds and tests. Continuous Deployment extends this by automatically deploying the tested code to a staging or production environment, such as an Apache Tomcat server.

2. Role of the Build Automation Tools

Build tools are responsible for transforming human-readable source code into executable artifacts. While multiple tools exist, the following three are the industry standards:

- Apache Maven: A project management and comprehension tool based on the Project Object Model (POM). It uses a pom.xml file to manage dependencies and follow a standard build lifecycle (compile, test, package, install, deploy).
- Gradle: A build automation system that builds upon the concepts of Ant and Maven but introduces a Groovy or Kotlin-based Domain Specific Language (DSL) instead of XML. It is noted for high performance via incremental builds and build caching.

3. Pipeline-as-Code (The Jenkinsfile)

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins. Defining the pipeline as a script (typically a Jenkinsfile) allows the build process to be versioned and reviewed alongside the application code.

The pipeline architecture is generally divided into two types:

- Declarative Pipeline: Provides a simplified, opinionated syntax with a pre-defined structure, making it ideal for standard CI/CD workflows.
- Scripted Pipeline: Utilizes a more robust Groovy-based syntax, offering maximum flexibility for complex logic and integrations.

4. Technical Stages of the Pipeline

The automated pipeline is logically partitioned into discrete "Stages." Each stage must complete successfully before the subsequent stage initiates.

A. Source Code Management (SCM)

   The process begins with the retrieval of the latest source code from a Version Control System (VCS) such as Git. Jenkins utilizes plugins to clone the repository into its local workspace. This stage ensures that the pipeline always operates on the most recent iteration of the codebase.

B. The Build and Compilation Stage

   During this phase, the chosen build tool (e.g., Maven) invokes the compiler. The source code is converted into Java .class files. Any syntax errors or dependency resolution issues at this stage will result in an immediate pipeline failure, providing rapid feedback to the development team.

C. Automated Testing

   Quality assurance is enforced through the execution of unit tests (using frameworks like JUnit or TestNG). The build tool executes these tests and generates reports. If the code fails to meet the defined assertions, the pipeline is halted to prevent the propagation of bugs.

D. Artifact Packaging

Once the tests are validated, the application is bundled into a deployable format. For Java web applications, this is typically a Web Archive (WAR) file. The WAR file contains the compiled classes, web resources (HTML/JSP), and necessary library dependencies.

E. Deployment to Apache Tomcat

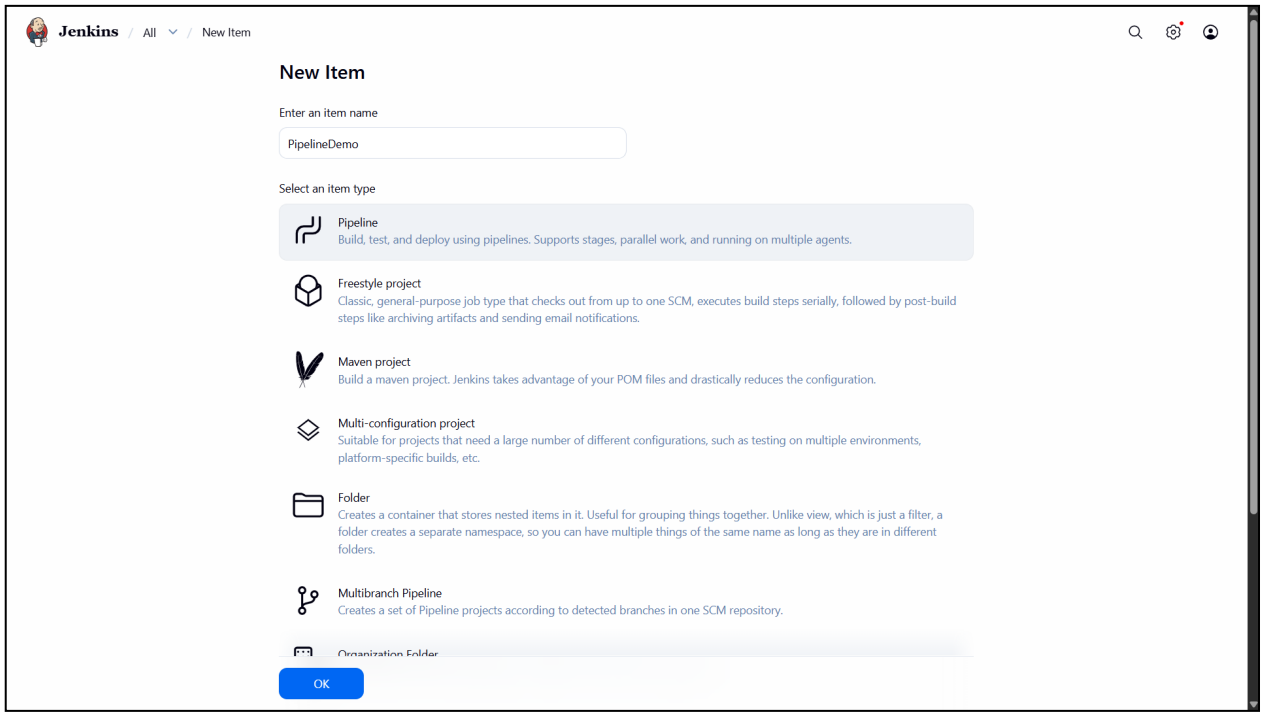
The final stage involves transferring the WAR file to the Apache Tomcat servlet container. This is achieved via the Tomcat Manager App API.

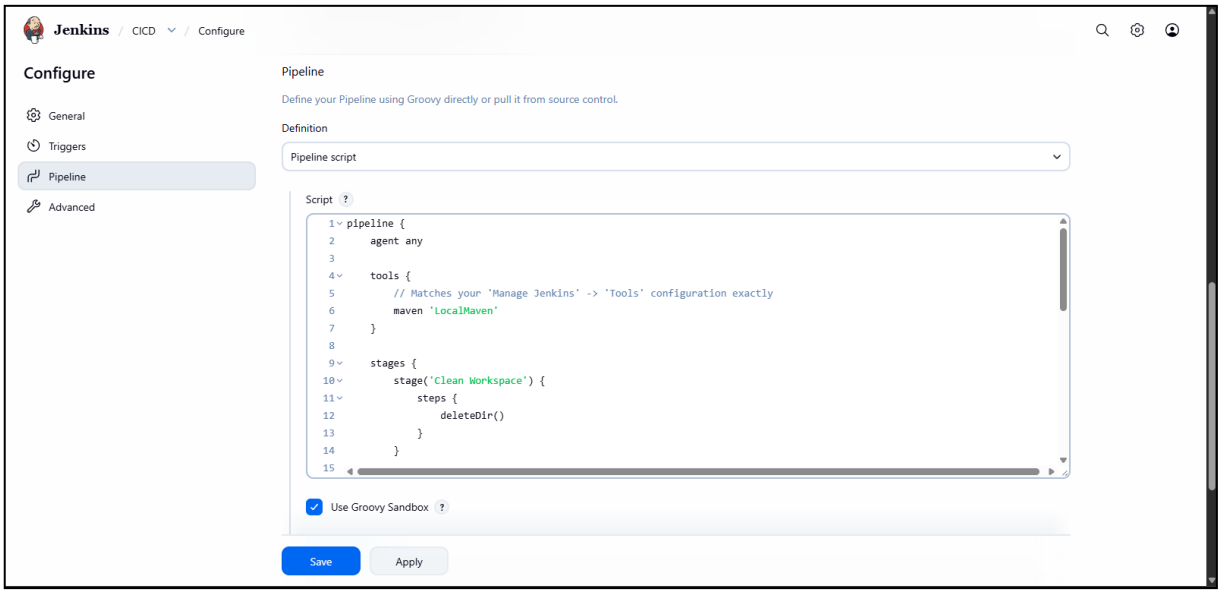
- Authentication: Jenkins must be configured with a set of credentials that possess the manager-script role.
- Execution: The pipeline communicates with the Tomcat server URL (e.g., http://<server-ip>:8080/manager/text) to undeploy the previous version and deploy the new artifact.

5. Architectural Components Summary

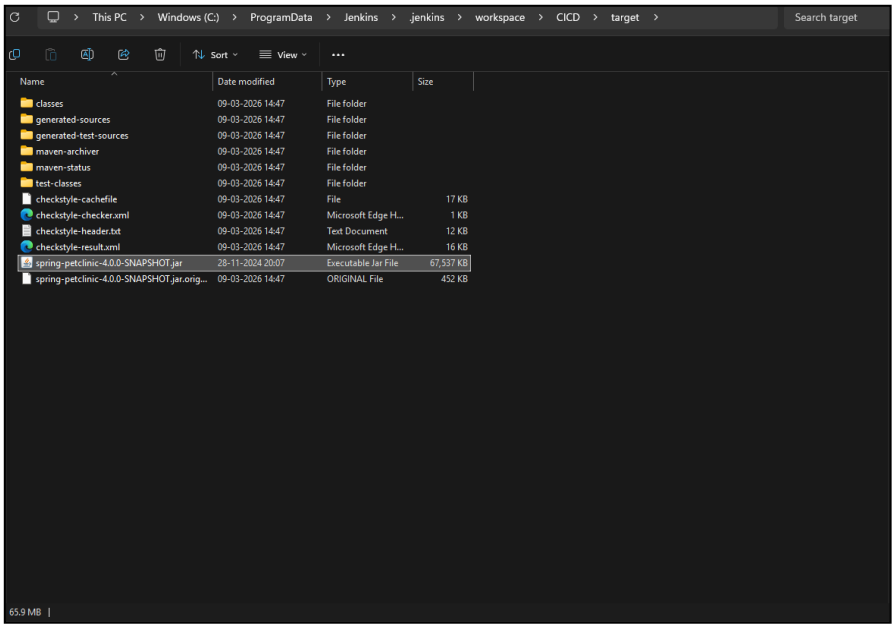
Component	Function
Jenkins	Orchestrates the workflow and manages execution logs.
Git / GitHub	Acts as the authoritative source of truth for the code.
Maven / Gradle	Handles dependency management and artifact generation.
JUnit	Ensures code reliability through automated assertions.
Apache Tomcat	Provides the runtime environment for the Java Web Application.

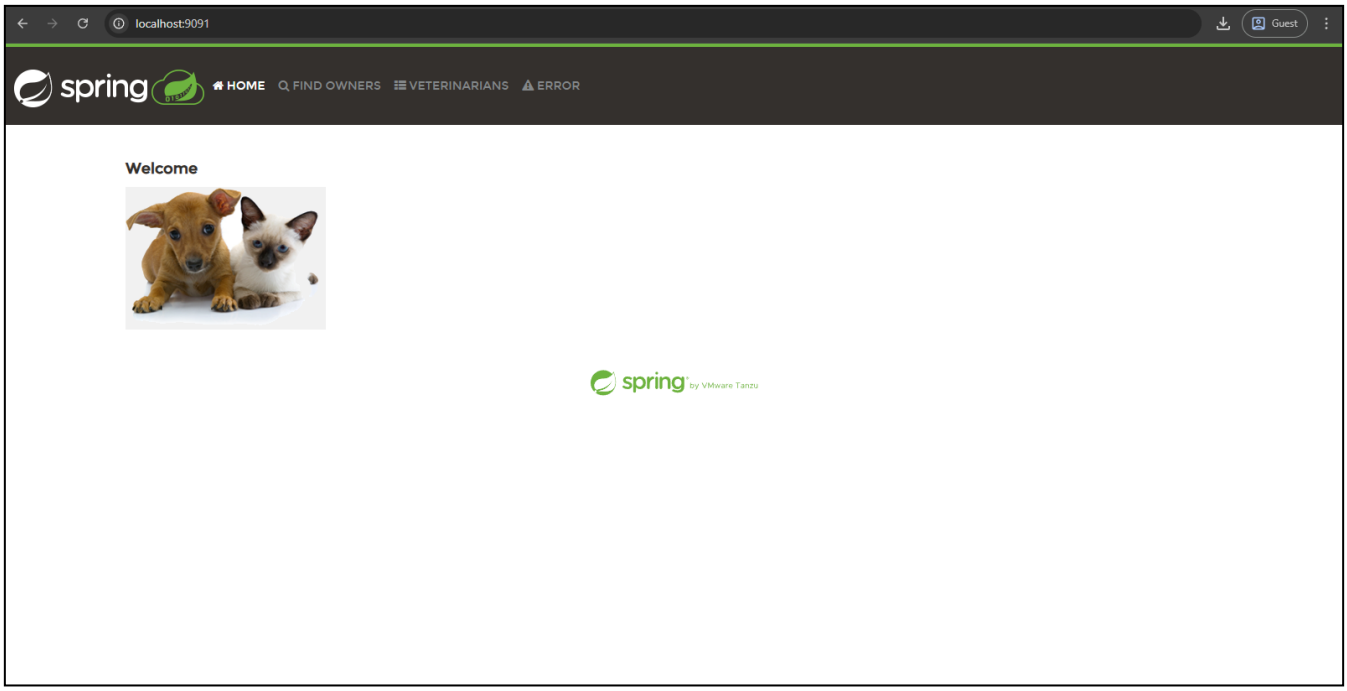
Implementation:


Jenkins / All / New Item
New Item
Enter an item name
PipelineDemo
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
OK


Jenkins / CICD / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
tools {
// Matches your 'Manage Jenkins' -> 'Tools' configuration exactly
maven 'LocalMaven'
}
stages {
stage('Clean Workspace') {
steps {
deleteDir()
}
}
Use Groovy Sandbox
Save
Apply

Jenkins / CICD / #21
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-lang3/3.14.0/commons-lang3-3.14.0.jar (658 kB at 1.6 MB/s)
[INFO] Building jar: C:\ProgramData\Jenkins\.jenkins\workspace\CICD\target\spring-petclinic-4.0.0-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:4.0.1:repackage (repackage) @ spring-petclinic ---
[INFO] Replacing main artifact C:\ProgramData\Jenkins\.jenkins\workspace\CICD\target\spring-petclinic-4.0.0-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\ProgramData\Jenkins\.jenkins\workspace\CICD\target\spring-petclinic-4.0.0-SNAPSHOT.jar.original
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 01:53 min
[INFO] Finished at: 2026-03-09T14:47:43+05:30
[INFO] --
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
Jenkins 2.541.2


This PC > Windows (C:) > ProgramData > Jenkins > .jenkins > workspace > CICD > target >
Search target
Sort
View
Name | Date modified | Type | Size
classes | 09-03-2026 14:47 | File folder
generated-sources | 09-03-2026 14:47 | File folder
generated-test-sources | 09-03-2026 14:47 | File folder
maven-archiver | 09-03-2026 14:47 | File folder
maven-status | 09-03-2026 14:47 | File folder
test-classes | 09-03-2026 14:47 | File folder
checkstyle-cachefile | 09-03-2026 14:47 | File | 17 KB
checkstyle-checker.xml | 09-03-2026 14:47 | Microsoft Edge H... | 1 KB
checkstyle-header.txt | 09-03-2026 14:47 | Text Document | 12 KB
checkstyle-result.xml | 09-03-2026 14:47 | Microsoft Edge H... | 16 KB
spring-petclinic-4.0.0-SNAPSHOT.jar | 28-11-2024 20:07 | Executable Jar File | 67,537 KB
spring-petclinic-4.0.0-SNAPSHOT.jar.orig... | 09-03-2026 14:47 | ORIGINAL File | 452 KB
65.9 MB



Conclusion:

Hence, we have successfully built the pipeline of jobs using Maven / Gradle / Ant in Jenkins, created a pipeline script to test and deployed an application over the tomcat server.

⌘⌘⌘⌘⌘⌘⌘